TRI10NWARE

Tritonware Game Jam
Theme: FLOW STATE

Consolidated game design / Updated October 12, 2026

Game description

The player is struck by lightning, unlocking the ability to enter **flow state**. Increased electrical current lets the player activate networks and use charged objects to extend their movement through the world.

A straightforward, logical precision platformer with pixel art and movement inspiration from *Celeste*. The central appeal is mechanical mastery: precise jumps, wall jumps, directional dashes, and state transitions, supported by planning a route through the room.

Game feel and design pillars

- **Precise, responsive execution.** Inputs and environmental interactions should be predictable; coyote time and jump buffering support the movement kit.
- **Strategically calculated clutch recoveries.** Players assess their route, resource budget, and recharge opportunities before committing to movement.
- **Complementary states.** Default offers useful footing on default-only platforms; flow enables electrical traversal and network activation. Each can matter within the same movement sequence.
- **Movement drives the challenge.** Combine switching with jumping, dashing, and positioning. Simple alternation can teach the rules; the intended depth comes from combining them.
- **Exploration supports mastery.** Interconnected rooms, backtracking, secrets, and optional challenges give players reasons to explore and improve.

Scope clarification
State switching does not change the player's literal physical properties. The earlier description of partial intangibility refers strictly to state-dependent platforms. Whole-wall phasing is outside the current design and, at most, a future possibility.

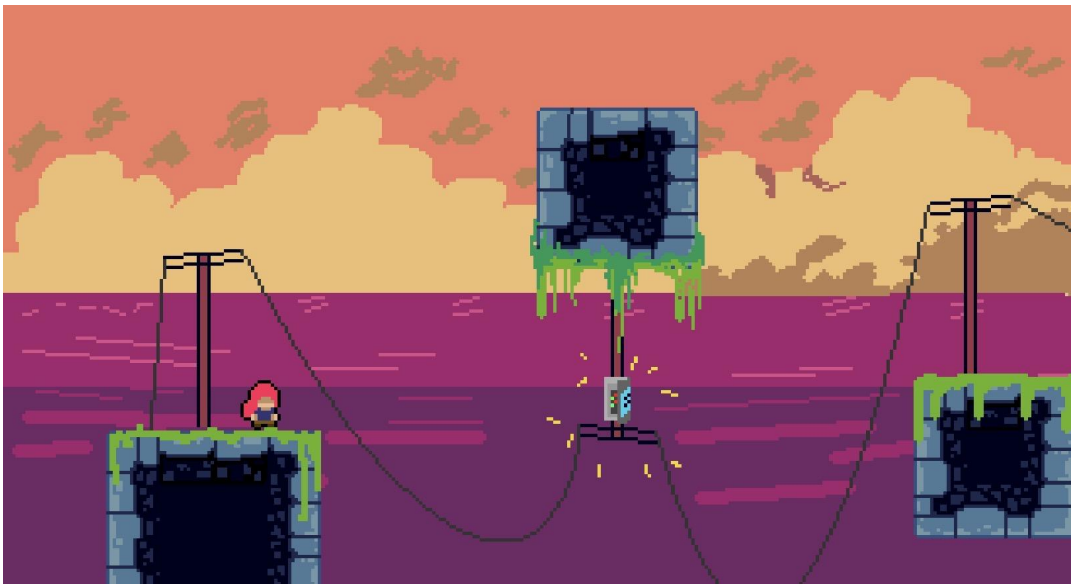
How to read this document

Current design records the user's selected direction. **Working assumptions** are provisional rules discussed but not fully specified. **Deferred questions** preserve details for prototype evaluation rather than inventing answers. **Future candidates** are notes, not first-prototype commitments.

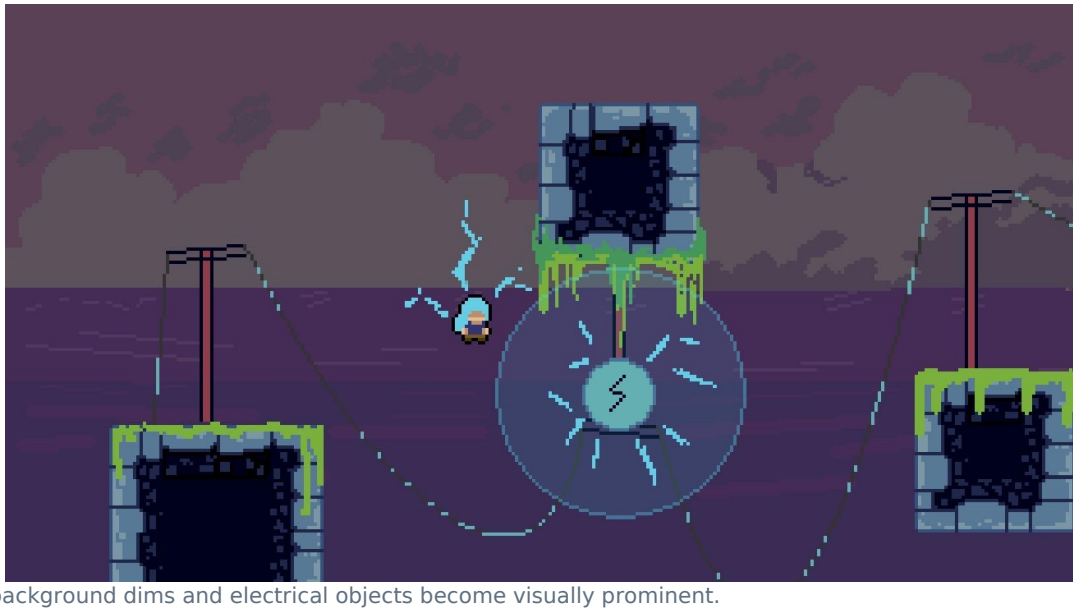
The original mockups are retained on page 2. Movement and states appear on pages 3-4; networks on page 5; volts and progression on page 6; level design on page 7; presentation and Godot notes on page 8; prototype checkpoints and deferred notes on page 9.

Core loop and original mockups

These original pixel-art mockups establish the visual direction and illustrate a charged-object crossing. They are concept references, not a complete specification of final level geometry.



Default: read the room, identify electrical objects, and plan the crossing.



Flow: the background dims and electrical objects become visually prominent.

Basic charged-object crossing

1. Analyze the gap and identify a charged object. **2.** Enter flow, spending 1 volt. **3.** Jump toward the object. **4.** While in valid contact/range, press jump again (spacebar in the original example) or use dash to extend traversal. **5.** Reach the destination and use default when a default-only landing requires it.

Network extension: When the route is unpowered, first reach and contact its source/root while in flow. Activation restores volts and powers connected objects, creating the next traversal path.

Movement and charged objects

Current movement kit

Ability	Current design
Jump	One ordinary jump; charged objects provide an additional aerial jump opportunity.
Wall jump	Jump from walls, drawing inspiration from *Celeste*.
Dash	Directional dash available in midair as well as on the ground; the player chooses direction.
Dash refresh	Touching the ground or successfully using a charged object refreshes dash availability.
Input support	Coyote time and jump buffering are part of the platformer design. Exact windows are tuning decisions.
Volt cost	Movement abilities do not consume health/volts. Entering flow is the state-transition cost.

Charged-object interaction

In flow, a powered object provides a movement opportunity when the player inputs jump or dash while landing on, contacting, or overlapping its valid interaction region. The purpose is to jump or dash farther, not merely to touch an object for a passive movement reward.

The object is reusable: it can grant **unlimited uses** when the player returns and performs valid interactions. It is not depleted after one jump and is not limited to one use until the player next touches the ground.

Terminology: Source contact activates a network. Charged-object *use* means executing a movement action from the object. These are separate events; mere proximity is not the intended dash-refresh reward.

Movement chains

Jump toward an object, execute a charged action, then use the refreshed dash to reach the next object. Returning to the same object is allowed. The boundary between a new encounter and repeated input within one region remains open.

Deferred interaction details
Define the interaction region, contact timing, input buffering, and whether leaving and re-entering the region is necessary before reuse. Also test jump versus dash behavior, refresh ordering, and input priority when both actions are pressed. Unlimited object reuse does not yet specify unrestricted repeated launches while staying inside its range.

Feel to establish in the prototype

Tune jump physics, gravity, acceleration, dash speed/duration, direction selection, landing recovery, and input windows through playtesting. Numerical values and state-dependent movement buffs are not established.

Default and flow: platform interactions

The two states change access to specific environmental interactions. Default should be useful during traversal, particularly for landing and positioning; flow provides network activation and charged movement. The goal is a complementary relationship, not two complete independent movement systems.

World element	Default	Flow
Ordinary ground and walls	Normal collision	Normal collision
Default-only platform	Supports the player	Player passes through it
Flow-only platform*	Player passes through it	Supports the player
Powered charged object	No charged movement interaction	Jump/dash interaction available
Unpowered network source	Does not activate network	Contact activates network

*Flow-only platforms were discussed as the complementary platform family. Their placement and inclusion in the first test room can be evaluated alongside default-only platforms. Ordinary walls do not phase with state.

Why return to default?

Returning to default can immediately provide useful footing on a default-only platform or allow passage through a flow-only platform. The default state does not need extra verbs, a separate movement kit, or a collection of physical puzzles to have value.

Switching as part of movement

When leaving a default-only platform for an electrical crossing, **jump first, then switch**. Switching while still standing on it removes its support. Midair transitions let players combine electrical movement with state-specific landings and passage through platforms.

The preferred challenge combines movement and switching: dash toward a charged object, change state to pass a state-dependent platform, then return to flow for the next charged action. Alternating platforms can teach this interaction, but the richer test is how transitions affect trajectory, timing, positioning, and volt use.

Platform collision questions to retain
Test the exact moment collision changes, how transitions behave during a dash, and what happens when switching would make an overlapping platform solid. Determine whether platforms are one-way or fully solid, and how to keep landing, passage, and rejected transitions readable. No overlap-resolution policy has been selected.

Ordinary world interactions

Levers, keys, buttons, and doors were considered as ways to expand engagement with the world. They are not a committed default-only system. State restrictions should serve a traversal purpose rather than merely add a toggle to an ordinary interaction. Altering or rearranging level geometry is not the envisioned jam design.

Networks: activation creates a path

Current activation rule

Physically contact the unpowered source/root while in flow state. Activation does not require a special dash impact or switching at a particular radius. A player who is already in flow can activate a root on contact; switching while physically touching it should follow the same contact rule.

Use one consistent rule. The source's position supplies the challenge: an accessible root can teach activation, while reaching a difficult root can require a precise movement sequence. Mechanical mastery comes from reaching it rather than adding a compulsory input to every activation.

Effects of activation

- Supply electricity to the connected network of objects, making its charged movement objects interactable.
- Restore the player's volts to the maximum of 3.
- Create a traversal path that was unavailable while the connected objects were unpowered.

The original concept connects objects through electrical networks, with A.C. units noted as part of the setting/object concept. The exact visual representation of roots, connections, and electrical objects remains an art and implementation decision.

Canonical pit-crossing example

An unpowered root is accessible on the player's starting side of a large pit. The player enters flow and contacts that root. The network powers the objects across the pit and restores volts. The player can now use those objects to cross. This is the selected network model; the player does not need to use an already powered object across the pit to turn the network on.

Working recharge rule

The discussion used a provisional rule that an already activated network cannot repeatedly restore volts through repeated contact. Treat activation and recharge as the same one-time event within the relevant reset period. Its exact scope across room visits, checkpoints, and death still needs definition; do not assume a finalized recharge-farming policy.

Reset and communication

Network activations reset on death; permanent progression remains. Show whether a network is powered, which objects belong to it, and what activation changed. The player should be able to plan a route and identify recharge opportunities before committing to a difficult crossing.

Deferred network questions
Define resets across connected rooms and checkpoints, recharge eligibility after resets, and whether the player can always reach a root from the side used for backtracking. Keep temporary power separate from permanent unlocks so resets do not trap the player.

Later network variants

Timed networks belong after players master normal activation. Mutually exclusive/interfering networks (A powered means B unpowered) remain an optional idea from the original document, only if time permits; they are outside the first prototype.

Volts, death, and permanent progress

Current resource rules

Rule	Design
Resource	Volts represent health; maximum 3.
Entering flow	Costs 1 volt per entry. Movement abilities do not have a volt cost.
Network activation	Restores all 3 volts.
Zero volts	Starts a death countdown, allowing an opportunity to reach a network and recharge.
Death and respawn	Fast respawn at the room-start/last checkpoint. Local challenge activations reset; permanent progression is retained.

The countdown replaces the original death-on-exiting-flow rule. The directional air dash also replaces the original ground-only restriction. Neither older restriction remains active.

Zero-volt details deferred for evaluation

- Does the countdown continue, pause, or stop after returning to default?
- Can the player re-enter flow at zero volts, and how is repeated switching handled during a recovery window?
- Should mandatory progression ever require spending the final volt, or is the countdown primarily recovery from mistakes?
- What is the countdown duration, how is it communicated, and precisely when does recharging cancel it?

A player at zero volts can recover by activating a network. Timer behavior and mandatory final-volt challenges remain questions for prototype evaluation.

Persistence layers

Layer	Current intent / detail to resolve
Temporary challenge state	Network activations reset on death. Define which room/network set resets.
Checkpoint state	Controls where the player respawns. Exact checkpoint placement and advancement rules remain open.
Permanent progression	Retained across death. Define which collectibles, unlocked passages, and world milestones count as permanent.

Recovery and backtracking checks

Check that resets, depleted volts, checkpoint changes, and approaching networks from the opposite side do not trap the player. Respawn volts, state, and dash availability remain unspecified prototype details.

Rooms, exploration, and rewards

Level structure

Rooms are interconnected and support backtracking and exploration. Room sizes vary to accommodate longer challenges rather than enforcing a single fixed size. Checkpoints and quick respawns should keep retries focused on learning the movement sequence.

Preferred challenge design

States should complement one another within traversal. Simply offering an ordinary route and an equally difficult flow route that costs health is not enough to make switching interesting. The chosen direction is to combine movement opportunities, platform interactions, and resource planning.

The preferred model is **Challenge B**: charged actions, dashes, jumps, and state transitions work together. A mandatory sequence can still be a satisfying execution challenge. It does not need an alternative route in every room; where strategic choice is intended, the choice should affect risk, positioning, switches, or resource use.

Useful room-design comparisons

- A default-only foothold can offer a stable intermediate landing, but reaching the next electrical object may require another flow entry.
- A more demanding movement sequence may bypass that foothold and conserve an entry. This is a comparison to test, not a required route structure.
- Place roots so their reachability and resulting network path are readable. Early rooms teach the contact rule; later placement can demand stronger execution.

Mandatory flow entries count against the volt budget. Check whether each sequence can be completed from its intended entry conditions and whether a recharge is reachable before resource exhaustion. Longer rooms and backtracking make these assumptions especially important.

Collectibles and optional challenges

Collectibles should encourage exploration **and** reward difficult movement challenges. Secrets and optional rooms can use the same movement and network rules. A currency economy or spending system has not been selected.

Defer the persistence choice: is a collectible retained immediately on pickup, or only after surviving/completing its challenge? Likewise, completion tracking and replay incentives remain possible uses, not settled requirements.

Feedback as a reward

Give successful jumps, charged actions, dashes, network activation, and room completion clear visual/audio feedback. Evaluate their feel before adding an economy or more systems.

Jam scope
Prototype the player kit, switching, state-dependent platforms, charged actions, and network activation first. Defer water, viruses, timed/interfering networks, geometry manipulation, and additional progression systems until the foundation is tested.

Pixel art, readability, and Godot notes

Visual direction

Use pixel art, as established by the original mockups. Keep the presentation readable and the world rules straightforward. Entering flow slightly dims the background and brings electrical objects into focus with glow/current effects.

- Hair color and the player sprite indicate default versus flow.
- A state-switch effect communicates the transition.
- In default, provide hints about which objects can become interactable in flow.
- Distinguish powered objects, inactive roots, and their network connections.
- Communicate state-dependent platform behavior, dash availability, volts, and a zero-volt countdown clearly enough to support planning.

Animation structure retained from the original

Original naming proposal: **default/electric + _ + idle/run/jump/dash** on AnimatedSprite2D. Use *flow* as the design term; *electric* can remain an asset naming term. A separate AnimationPlayer or layered effect for state-switch VFX was proposed so it can play alongside the movement animation.

Original Godot rendering candidates

Candidate	Purpose / status
CanvasLayer	Possible dimming overlay. Check layer order so the player and relevant objects remain readable; an overlay can otherwise cover everything.
CanvasModulate	Alternative world-color/dimming control to evaluate.
WorldEnvironment (Canvas)	Original candidate for glow; rendering support and setup need verification in the chosen project.
TileMapLayer light occlusion	Original note for controlling how lighting relates to level geometry.
Directional light	Optional level-lighting experiment.
Point-light children on objects	Original suggestion for the player, electrical objects, coins/collectibles, and other object scenes.

These are retained implementation ideas, not a requirement to use every system together. Select the approach that achieves the mockup's contrast and readable pixel-art presentation after testing it in the project.

Onboarding

Introduce the ordinary movement kit and the visual state cues, then a simple charged crossing and an accessible network root. Teach state-dependent platforms through observable movement consequences. More demanding chains, optional challenges, and any later network variants should build on the same consistent rules.

Prototype checkpoints and deferred notes

First prototype: prove the movement foundation

Build a compact test space using the player kit, default/flow switching, state-dependent platforms, reusable charged objects, and a contact-activated network. Evaluate whether chaining these interactions feels fluid and whether the player can anticipate the consequences of each action.

Checkpoint	What to assess
Movement feel	Are jump, wall jump, directional dash, coyote time, and input buffering responsive and predictable?
Charged chains	Does input during valid contact reliably trigger the intended jump/dash and refresh? Test reuse of the same object and sequences of objects.
Complementary states	Does returning to default provide useful footing/passage, and does switching combine meaningfully with trajectory and positioning?
Network readability	Can players identify the root, activate it through flow contact, and understand the path and recharge it creates?
Resources and retries	Test the final volt, countdown recovery, repeated contact, death reset, respawn, and revisiting a room from the opposite side.

Questions retained for after the prototype

Revisit platform overlap and collision timing; charged-object reuse/re-entry and input priority; zero-volt switching and timer behavior; recharge/reset scope; checkpoint and permanent-unlock boundaries; collectible persistence; and visual/audio feedback. Where a behavior is needed to run the prototype, use an explicit temporary rule and evaluate it rather than presenting it as a settled design decision.

Also retain world-consistency questions: how state-dependent platforms are explained and communicated, whether a later state-specific hazard feels logical, and whether switching remains interesting without relying entirely on the volt economy. These questions need not delay the prototype or require a change to the player's physical properties.

Future candidates: notes only

- **Water:** Possible environment that is safe in default and cannot be safely entered in flow. No water/swimming physics is intended; the exact hazard consequence remains open.
- **Propulsion objects:** The repelling-surface idea could become a separate charged object that propels the player a long distance vertically or horizontally. Not part of the initial prototype.
- **Viruses:** No flow-only enemies in the first prototype. Original concept: a virus travels along a network and is visible only in flow. Later discussion: activating an infected network could spawn a tracking enemy. Possible progression from a flow-visible virus to a type present in both states; possible future story-theme connection. Record only; no story development is included.
- **Other later possibilities:** Timed networks after mastery, interfering networks if time permits, and whole-wall phasing at most as a future option.

For later viruses, retain the questions of whether switching trivializes danger and whether a hazard can make a necessary charged crossing unfair or impossible. Visibility, existence, and collision across states are separate rules that have not been finalized.